



# 6.1. Programació Modular

## RA2

MP0485 - Programació

Curs 2025-2026



# 1. Què és un script

Un **script** és un codi creat per realitzar una tasca concreta.

## Característiques habituals:

- Es troba en un únic mòdul de *Python* (fitxer .py).
- Defineix diferents funcions senzilles.
- Fa ús de llibreries del sistema i comandes de bash.
- S'executa des de la consola.
- Fa ús d'arguments des de consola (ho treballarem més endavant).
- S'integra amb facilitat amb tasques de tipus cron (periòdiques, automatitzades).



## 2. Estructura d'un programa

A mesura que avancem, els **programes** cada vegada són més llargs i complexos i, en general, executen diverses vegades **trossos de codi iguals** o molt semblants.

**Exemple:** Programa que demana dos valors de temperatura i humitat per fer la mitjana.

```
1  try:
2      t1 = float(input("Temperatura matí: "))
3  except ValueError:
4      print("Valor no vàlid. Assigno 0.")
5      t1 = 0.0
6
7  try:
8      t2 = float(input("Temperatura tarda: "))
9  except ValueError:
10     print("Valor no vàlid. Assigno 0.")
11     t2 = 0.0
12
13 print("Mitjana temperatura:", (t1 + t2) / 2)
14
15 try:
16     h1 = float(input("Humitat matí: "))
17 except ValueError:
18     print("Valor no vàlid. Assigno 0.")
19     h1 = 0.0
20
21 try:
22     h2 = float(input("Humitat tarda: "))
23 except ValueError:
24     print("Valor no vàlid. Assigno 0.")
25     h2 = 0.0
26
27 print("Mitjana humitat:", (h1 + h2) / 2)
28
```



## 2. Inconvenients

Ens trobem amb **programes amb codi...**

- molt llarg
- poc llegible
- no reutilitzable
- complicat de mantenir/modificar

I tot això, ens **complica el fet de programar:**

- dificultat a l'hora de comprendre que fa un codi, ja sigui al moment o temps després.
- lentitud al haver de repetir les coses i modificacions.

```
1  try:
2      t1 = float(input("Temperatura matí: "))
3  except ValueError:
4      print("Valor no vàlid. Assigno 0.")
5      t1 = 0.0
6
7  try:
8      t2 = float(input("Temperatura tarda: "))
9  except ValueError:
10     print("Valor no vàlid. Assigno 0.")
11     t2 = 0.0
12
13 print("Mitjana temperatura:", (t1 + t2) / 2)
14
15 try:
16     h1 = float(input("Humitat matí: "))
17 except ValueError:
18     print("Valor no vàlid. Assigno 0.")
19     h1 = 0.0
20
21 try:
22     h2 = float(input("Humitat tarda: "))
23 except ValueError:
24     print("Valor no vàlid. Assigno 0.")
25     h2 = 0.0
26
27 print("Mitjana humitat:", (h1 + h2) / 2)
28
```

Està repetit!



### 3. Mètodes

Un **mètode** és un **petit subprograma** que s'encarrega de **fer una tasca** dins del programa.

→ Els **mètodes** ens permeten empaquetar codi, donar-li un nom, i utilitzar-lo “*cridant-lo*” (escrivint el nom que li hem donat).



**Procediments:** fan una tasca i no retornen cap valor.

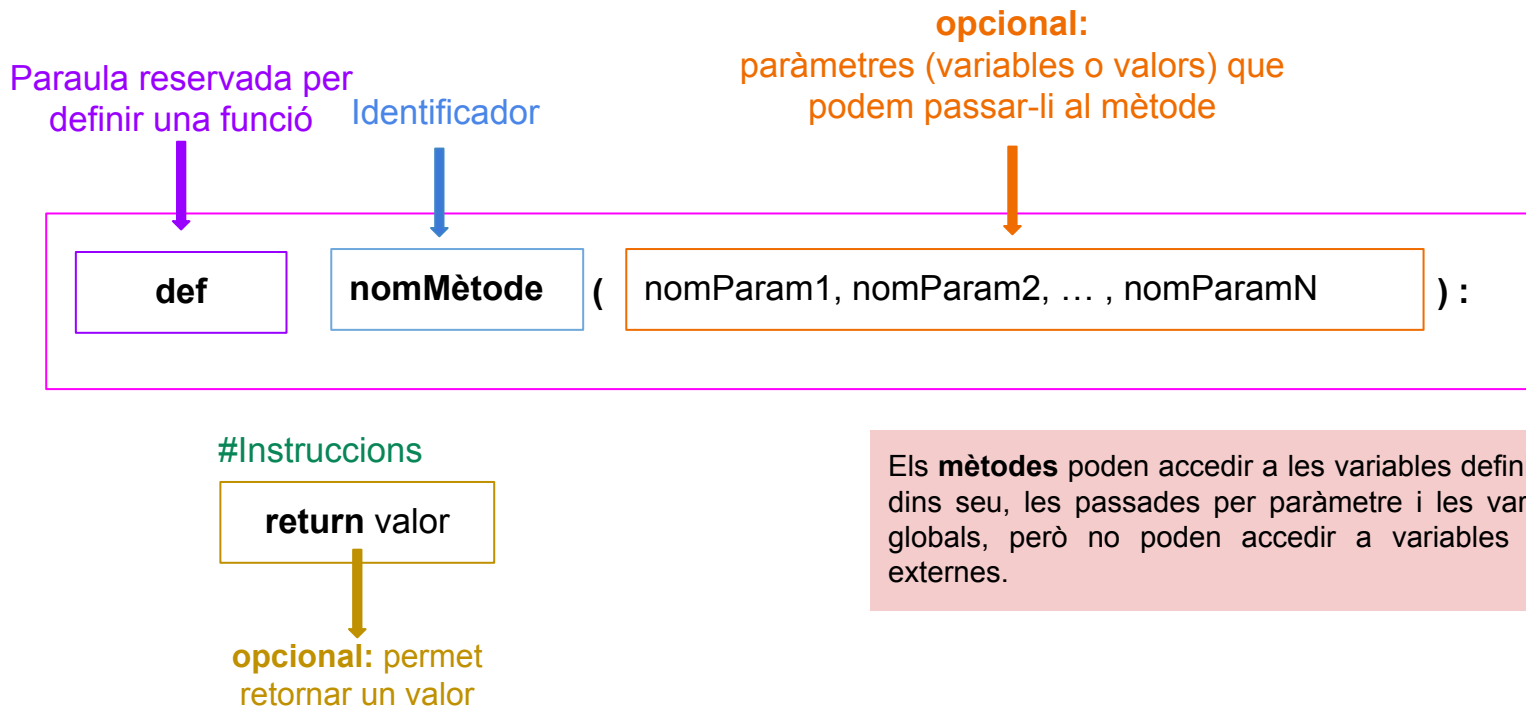
**Exemple:** imprimir un missatge

**Funcions:** fan una tasca i retornen un valor.

**Exemple:** realitzar una suma.



## 4. Sintaxi



Els **mètodes** poden accedir a les variables definides a dins seu, les passades per paràmetre i les variables globals, però no poden accedir a variables locals externes.



# 5. Procediments vs Funcions

**1. Procediments:** fan una tasca i no retornen cap valor.

**Exemple (sense paràmetres):** saludar per pantalla

```
def dirHola():  
    print("Hola! Joan")
```

**Exemple (amb paràmetres):** imprimir una llista per pantalla.

```
def dirHola(nom):  
    print(f"Hola! {nom}")
```

**2. Funcions:** fan una tasca i retornen un valor.

**Exemple (sense paràmetres):** demanar valor

```
def preguntarEdat():  
    print("Quina edat tens?")  
    edat = int(input("Introdueix la teva edat: "))  
    return edat
```

**Exemple (amb paràmetres):** realitzar una suma

```
def sumar(valor1, valor2):  
    suma = valor1+valor2  
    return (valor1+valor2)
```

## 6. Exemple

Vull fer un programa que **demana dos valors de temperatura i humitat per fer la mitjana**.

```
1 try:
2     t1 = float(input("Temperatura matí: "))
3 except ValueError:
4     print("Valor no vàlid. Assigno 0.")
5     t1 = 0.0
6
7 try:
8     t2 = float(input("Temperatura tarda: "))
9 except ValueError:
10    print("Valor no vàlid. Assigno 0.")
11    t2 = 0.0
12
13 print("Mitjana temperatura:", (t1 + t2) / 2)
14
15 try:
16     h1 = float(input("Humitat matí: "))
17 except ValueError:
18     print("Valor no vàlid. Assigno 0.")
19     h1 = 0.0
20
21 try:
22     h2 = float(input("Humitat tarda: "))
23 except ValueError:
24     print("Valor no vàlid. Assigno 0.")
25     h2 = 0.0
26
27 print("Mitjana humitat:", (h1 + h2) / 2)
28
```

sense mètodes

```
1 def demanar_float(text):
2     try:
3         return float(input(text))
4     except ValueError:
5         print("Valor no vàlid. Assigno 0.")
6         return 0.0
7
8 def mitjana(a, b):
9     return (a + b) / 2
10
11 t1 = demanar_float("Temperatura matí: ")
12 t2 = demanar_float("Temperatura tarda: ")
13 print("Mitjana temperatura:", mitjana(t1, t2))
14
15 h1 = demanar_float("Humitat matí: ")
16 h2 = demanar_float("Humitat tarda: ")
17 print("Mitjana humitat:", mitjana(h1, h2))
18
```

amb mètodes







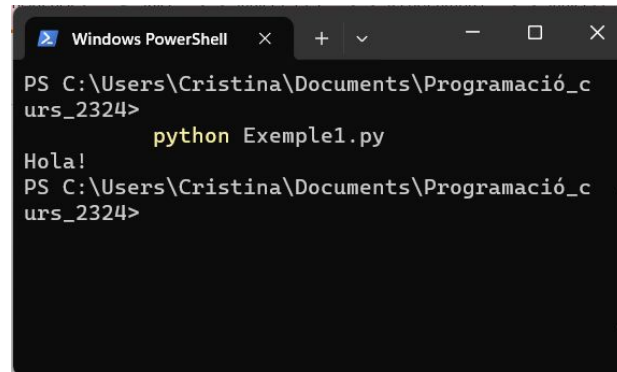
## 7. Estructura: `__main__`

Quan un arxiu de *Python* s'executa, l'interpret assigna el nom `'__main__'` a l'script actual.

Això no passa si s'importa l'arxiu a un altre script.

- La comprovació `"__name__ == '__main__'"` ens permet saber si el mòdul s'està executant o important, de forma que només s'executi l'script quan cal.
- Això permet la reutilització de codi/mètodes que puguem necessitar fàcilment.

```
if __name__ == '__main__':  
    print("Hola!")
```



```
Windows PowerShell  x  +  v  -  □  x  
PS C:\Users\Cristina\Documents\Programació_c  
urs_2324> python Exemple1.py  
Hola!  
PS C:\Users\Cristina\Documents\Programació_c  
urs_2324>
```



# 7. Exemple

Script1.py

Script2.py

```
def demanar_dades():
    usuari = input("Nom d'usuari: ")
    contrasenya = input("Contrasenya: ")
    return usuari, contrasenya

def validar_inici_sessio(usuari, contrasenya):
    if usuari == "admin" and contrasenya == "1234":
        return True
    else:
        return False

def main():
    print("=== Inici de sessió (Script2) ===")
    usuari, contrasenya = demanar_dades()
    if validar_inici_sessio(usuari, contrasenya):
        print(f"Inici de sessió correcte! Benvingut/da {usuari}")
    else:
        print("Nom d'usuari o contrasenya incorrectes.")

main()
```

```
def demanar_dades():
    usuari = input("Nom d'usuari: ")
    contrasenya = input("Contrasenya: ")
    return usuari, contrasenya

def validar_inici_sessio(usuari, contrasenya):
    if usuari == "admin" and contrasenya == "1234":
        return True
    else:
        return False

def main():
    print("=== Inici de sessió (Script2) ===")
    usuari, contrasenya = demanar_dades()
    if validar_inici_sessio(usuari, contrasenya):
        print(f"Inici de sessió correcte! Benvingut/da {usuari}")
    else:
        print("Nom d'usuari o contrasenya incorrectes.")

if __name__ == "__main__":
    main()
```



```
from Script1 import demanar_dades, validar_inici_sessio

def main():
    print("=== Inici de sessió (Script3) ===")
    usuari, contrasenya = demanar_dades()

    if validar_inici_sessio(usuari, contrasenya):
        print("Accés concedit! Benvingut/da,", usuari)
    else:
        print("Accés denegat. Si us plau, torna-ho a provar.")

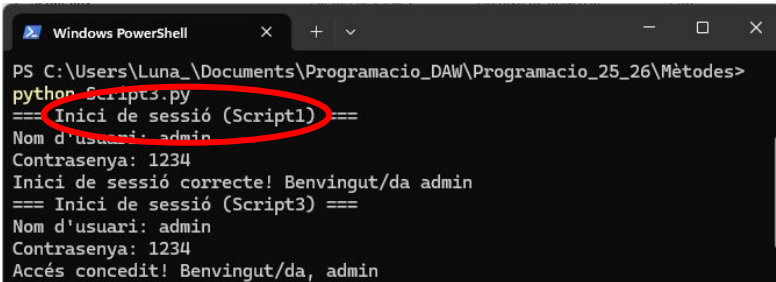
if __name__ == "__main__":
    main()
```

```
from Script2 import demanar_dades, validar_inici_sessio

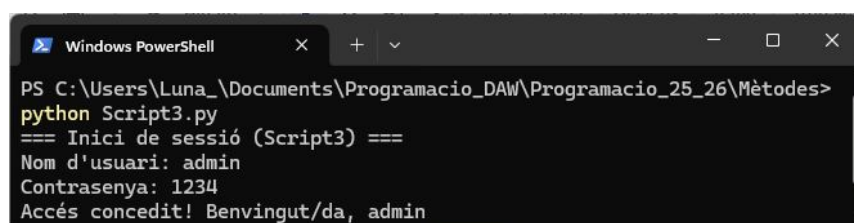
def main():
    print("=== Inici de sessió (Script3) ===")
    usuari, contrasenya = demanar_dades()

    if validar_inici_sessio(usuari, contrasenya):
        print("Accés concedit! Benvingut/da,", usuari)
    else:
        print("Accés denegat. Si us plau, torna-ho a provar.")

if __name__ == "__main__":
    main()
```



```
Windows PowerShell
PS C:\Users\Luna_\Documents\Programacio_DAW\Programacio_25_26\Mètodes>
python Script3.py
=== Inici de sessió (Script1) ===
Nom d'usuari: admin
Contrasenya: 1234
Inici de sessió correcte! Benvingut/da admin
=== Inici de sessió (Script3) ===
Nom d'usuari: admin
Contrasenya: 1234
Accés concedit! Benvingut/da, admin
```



```
Windows PowerShell
PS C:\Users\Luna_\Documents\Programacio_DAW\Programacio_25_26\Mètodes>
python Script3.py
=== Inici de sessió (Script3) ===
Nom d'usuari: admin
Contrasenya: 1234
Accés concedit! Benvingut/da, admin
```



## 9. Referències

<https://j2logo.com/python/tutorial/funciones-en-python/>

<https://j2logo.com/tipo-parametros-funcion-python/>

<http://elclubdelautodidacta.es/wp/2011/12/python-capitulo-27-funciones-con-argumentos-por-defecto/>

<https://docs.hektorprofe.net/python/programacion-de-funciones/paso-por-valor-y-referencia/>

□ **CC BY 4.0**

Contingut elaborat per Carles Bonet Papell i Cristina Reina León.

<https://creativecommons.org/licenses/by/4.0/>